

Strata — Developer Guide

For engineers implementing tickets

strata-kb v1.4.0 · 2026-09-19

1. Who this guide is for

You write product code. Tickets arrive already grounded in the organisation's knowledge base, with citations pinned to a specific version of a specific document, and you implement them.

Strata does two things for you.

It tells you what the source actually said. Not "check the handbook" — the exact section, at the exact revision the analyst was reading, with a warning if it has changed since.

It publishes what your codebase is, so everyone else — analysts drawing architecture diagrams, other teams, future you — can search your services, dependencies, endpoints and schema without reading your repository.

1.1 What a dev repo does and does not do

Does	Does not
Read the hub's published knowledge	Ingest documents from outside the repo
Publish knowledge about its own code	Publish domain knowledge
Run a five-command implementation workflow	Merge or ship anything by itself

The boundary is deliberate. A `child` repository brings in outside documents; a `dev` repository ingests nothing and authors only knowledge about itself. That is what makes it safe for your CI to publish continuously.

1.2 The two documents your repo publishes

Document	Origin	Regenerated	Needs review
<code><repo_id>-code</code>	<code>kb code-ingest</code> — deterministic, no model	on demand, from the working tree — never committed; CI publishes it on every push to the default branch	no
<code><repo_id>-svc</code>	drafted by a model from that evidence, corrected by you	never	always

`-code` is machine fact: service names, technologies, dependencies, commands, endpoints, schema, folder structure.
`-svc` is human claim: what each service is actually *responsible for*.

They are joined by section id — `svc.<name>` exists in both — so an analyst drawing a C4 container gets the alias, label and technology from `-code` and the description from `-svc`.

2. Setup, once

```
pip install strata-kb
kb init --kind dev
```

2.1 Configure

```
# .kb/config.yaml
kind: dev
repo_id: payments
hub: https://github.com/acme/kb-hub.git
intake: https://kb-intake.acme.com
```

- **hub** is the only read source for `kb query`, `kb get`, `kb resolve` and MCP.
- **intake** is how CI publishes. Ask the hub maintainer to add this repository under `repos:` in the hub's `federation/registry.yaml` — publishing is refused until they do.

If the KB should carry SQLite schema, or extra index tags, say so here — not in `kb-code.yaml` — so every local refresh and CI ingest the same way:

```
code_ingest:
  db: [data/app.sqlite] # explicit only
  tags: [payments]
```

2.2 Connect your assistant

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) were scaffolded already wired to the hub's MCP tools, but through placeholders — `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}` — that nothing has set yet.

Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`). It asks for the hub's HTTP base URL and token, writes both into `.env`, makes sure git ignores that file, and then verifies them against the hub, so a wrong URL and a rejected token come back as different errors:

Variable	Example
<code>STRATA_KB_HUB_URL</code>	<code>http://kb-hub.example.com:8321</code>
<code>STRATA_KB_HTTP_TOKEN</code>	the hub's bearer token

A bare re-run verifies again without retyping the token. Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` environment variable over `--token` — that flag leaves the token in your shell history.

`.mcp.json` and `.cursor/mcp.json` read those two variables from your environment, not from `.env` directly, so load `.env` into your shell (`set -a; source .env; set +a`, or use `direnv`) and restart your assistant — MCP reads the environment only at startup.

2.3 Turn on the PR gate

`kb init` writes `.github/workflows/kb-pr-lint.yml` but cannot enable branch protection for you. Add `pr-lint` to the branch's required checks.

Decide this knowingly. Unlike the BA repo's gate, this one never self-skips. Once required, it blocks **every** pull request lacking the required sections — including bot pull requests such as a dependency bump or a revert.

2.4 Ask about the hub's auto-merge policy

`-code` pull requests on the hub may be auto-merged; `-svc` pull requests must never be. Ask your hub maintainer how it is configured, and make sure of one thing:

`kb ci-publish` opens **one** hub pull request per push, not one per document. The intake keys the publish branch by repository id and reuses it while a PR is pending — so a push carrying a regenerated `-code` alongside an already-merged `-svc` amend lands **both** in that same PR.

The auto-merge rule must therefore be **path-scoped to** `.kb/<repo_id>-code/**`, never a whole-PR or whole-repo rule. A rule that fires on "this is a dev repo's `-code` publish" silently defeats the `-svc` review gate the two-document split exists to protect.

This auto-merge rule is also what keeps the hub from lagging main: without it, every push adds to one pending hub PR that someone has to merge. `kb doctor` reports when the hub's `-code` lags.

3. Onboarding an existing project

Your repository already has services doing real work. Adopting Strata should not mean documenting the system from a blank page, so there is a one-time seed.

`/dev-code-seed`

Seven steps:

#	Step	Who
1	Preflight — confirms hub, <code>repo_id</code> and <code>intake</code> are set and the repo is allowlisted on the hub; warns on a dirty tree	agent
2	Extract — <code>kb code-ingest --scaffold-svc</code> regenerates <code>-code</code> and creates a pending <code>svc-<name></code> scaffold per detected service, backed by deterministic code evidence	agent
3	Draft — <code>kb summarize <repo_id></code> writes a responsibility paragraph for each from that evidence	agent
4	Review — walk every drafted section against its evidence and correct it	you
5	Approve — <code>kb approve <repo_id>-svc</code> tips corrected sections to <code>reviewed</code>	you
6	Flows (optional) — add <code>flow-<name></code> sections for business flows crossing several services	you
7	Validate and publish — <code>kb build</code> without <code>--allow-pending</code> , then <code>kb publish --pr</code>	you

3.1 Step 4 is the actual work

The model's draft is a draft, not a fact. It can name the wrong flow, miss a responsibility, or over-claim one. Approving a section you have not read defeats the entire point of the gate.

Read each `svc.*` section's summary next to its code evidence and correct it. Where a responsibility touches a domain standard, cite `doc-id $section` from the hub instead of restating the rule in your own words.

Budget 10–15 minutes per service. That is the honest cost, and it is a one-time cost. After the seed, `-code` regenerates itself and `-svc` accrues a few lines at a time.

For step 6, skip freely. A missing flow beats a guessed one.

3.2 Why the first publish is `--pr` by hand

The seed happens *before* anything has merged to the default branch, so there is no push for `kb-code.yml` to react to.

Use `kb publish --pr`. Plain `kb publish` would tag the commit and poll the intake for up to ten minutes waiting on a tag-triggered CI run this repository does not have, then fail with a misleading error.

That hub pull request is reviewed by an analyst or architect, like any other hub content. `-svc` is curated knowledge; it publishes through review, never auto-merge.

4. Implementing a ticket

Five commands, scaffolded for Claude Code, GitHub Copilot and Cursor.

```
/dev-implement-ticket <ticket>      intake · resolve · ground · placeholders
├── dev-design      — A1: independent reviewer — GATE 1: you approve the design
├── dev-plan        — A2: independent reviewer — GATE 2: you approve the plan
├── dev-execute     A3 per task · A4 at branch end (repeatable, resumable)
└── dev-handover    — A5: merge-risk review — GATE 3: you open the PR
                        GATE 4: you merge it
```

`/dev-implement-ticket` is also the orchestrator: given a fresh ticket it runs the first four steps, then invokes the phases in order, detecting how far a ticket already got so it never redoes finished work.

4.1 Pick your entry point

Situation	Run
New ticket, nothing started	<code>/dev-implement-ticket <ticket></code>
Small ticket, the change is obvious	<code>/dev-implement-ticket <ticket></code> — the flow collapses itself; the design is still written, just a bounded one
Design approved, no plan yet	<code>/dev-plan <id></code>
Plan approved, or execution in progress	<code>/dev-execute <id></code>
Code hand-implemented, needs a PR write-up	<code>/dev-handover <id></code>
Lost track of where a ticket stands	<code>/dev-implement-ticket <id></code>
Just checking a citation	<code>kb resolve <file></code>

Every entry point re-checks citation freshness first. The hub may have published since your last session, so a reference that was `ok` yesterday can be `stale` today. Checking only at handover would be too late.

4.2 Where work lives

File	Written by	Purpose
<code>docs/impl/<id>-design.md</code>	<code>dev-design</code>	the design
<code>docs/impl/<id>-plan.md</code>	<code>dev-plan</code>	one task per acceptance criterion, <code>- []</code> checkboxes, a <code>Depends on:</code> line per task
<code>docs/impl/<id>-context.md</code>	<code>dev-implement-ticket</code>	resolved-context cache; gitignored, regenerated on demand
<code>docs/impl/<id>-review/</code>	<code>dev-execute</code> , <code>dev-handover</code>	per-task diffs and reviewer reports, the branch diff, the merge-risk report; gitignored, derivable from git and the review records below

The design and plan files each grow a `## Review record` table as their reviews run — the committed history of every round, not a separate file.

State is derived from the first two files, the current branch, and whether a PR is open — **never from conversation history**. Any phase resumes cold in a brand new session.

`kb init` never touches your `docs/impl/` content. It only adds `.gitkeep` and a `.gitignore` so the context cache and the review artefacts never land in a pull request.

Waves and lanes. Every task in the plan carries a `Depends on:` line under its `### Task <n>:` heading — `none`, or the tasks whose Interfaces it consumes or whose files it shares. `dev-plan` derives it; the A2 reviewer checks it. `kb plan waves docs/impl/<id>-plan.md` turns those lines into waves (wave `n` = tasks whose dependencies all sit in earlier waves) and fails when two tasks share a file without a dependency between them, when there is a cycle, or when a line is missing — a plan like that has no safe parallel order. A plan where every task lacks the line predates waves and runs sequentially.

`dev-execute` runs it after isolating the branch. A wave of one task runs as before; a wave of several runs each task in its own git worktree lane cut from the ticket branch, at most 3 lanes at a time, merged back in task order with `--no-ff`, the full suite run once per wave. A merge conflict means the plan was wrong and goes back to `dev-plan`.

4.3 Four human gates, five agent review rounds

Nothing in this pipeline merges or ships without a human.

1. **Design approved** — you sign off before any plan is written.
2. **Plan approved** — you sign off before any code is written.
3. **PR opened** — the agent writes the body; you open it.
4. **Merged** — you review and merge.

The agent does neither of the last two itself.

Before each of the first three gates, a different agent has already read the work back in a context of its own — never the same agent that produced it:

	Where	What it looks at
A1	<code>dev-design</code> , before GATE 1	whether the design matches the ticket
A2	<code>dev-plan</code> , before GATE 2	whether the plan is test-first and runnable
A3	<code>dev-execute</code> , per task	whether this task's diff meets its spec, and whether the code is good
A4	<code>dev-execute</code> , after the last task	whether the finished branch rounds out the ticket
A5	<code>dev-handover</code> , before GATE 3	whether merging into the default branch is safe

Criteria live in `docs/pr-review-rubric.md`, overridden by `docs/pr-review-rubric.local.md`. Each round allows at most 3 fix/review passes. A round only counts as clean once no BLOCKER and no SUGGESTED finding is left open — NOTE and NITS get recorded, not fixed — and a BLOCKER or SUGGESTED still standing after round 3 stops the flow and hands it to you.

Where your assistant's runtime cannot dispatch subagents — the default for the Copilot and Cursor wrappers — the review still runs, but as a single pass reading only what it was handed, and it is a **report**, not a **gate**: the phase stops there and hands the result to you instead of advancing on its own say-so.

5. What is enforced, and what is only asked

Be clear-eyed about the difference.

5.1 Machine-enforced

Rule	Enforced by
The PR carries its evidence	<code>kb pr lint</code> in <code>kb-pr-lint.yml</code> on every pull request
Unreviewed knowledge cannot reach the hub	<code>kb build</code> exits 1 while any <code>-svc</code> section is <code>pending</code>
The context cache is CLI-owned	<code>kb resolve --status-only --cache</code> refuses a cache whose version, ref set or resolved block differs from the ticket's
The merge-risk review has run and no BLOCKER remains	<code>kb pr lint</code> — the <code>## Review</code> section must carry a <code>Blocking: No</code> line

`kb pr lint` fails when a required section is missing, still holds the template's comment, claims verification with no pasted output, has no Verification fence containing the plan's test command, or when `## TDD exemptions` names a class outside `config`, `ci`, `docs`, `style`.

5.2 Prompt-only — stated in every wrapper, measured by nothing

These are rules the skills repeat. Nothing in `kb` enforces or measures them. The only check is the human at the gate — which means you.

- **TDD.** No production code without a failing test observed first, at every step of `dev-execute`. Exempt classes are named in `docs/tdd-exemptions.md` and declared in the plan.
- **Shown verification.** No completion claim without the real command output. `kb pr lint` checks the PR body, not the session.
- **Pinned values, verbatim.** Every standard-derived value comes from the resolved section at its pinned hub version, with a citation comment.
- **The ticket is read-only.** Findings go back to the analyst; the agent never edits the ticket.
- **Gates 1 and 2.** Design and plan approved before the next phase. The `status:` header records it; a human grants it.
- **OPEN (BA)**. An ambiguous acceptance criterion is escalated, never reinterpreted.
- **A1–A4.** The four agent reviews before GATE 3 are prompt discipline. Nothing in `kb` measures whether they ran. Only A5 has teeth: `kb pr lint` reads the PR's `## Review` section. An agent that skips A1–A4 and writes an honest `Blocking: No` still passes — the gate sits where the merge actually happens, not at every step before it.

6. Working with citations

```
kb resolve tickets/ABC-123.md # ok | stale | broken, per ref
kb diff hr-handbook --against <rev>
kb get hr-handbook 4.12 --level 13
```

Status	Meaning	What to do
ok	The pinned content still matches what is published	Implement against it
stale	The source was amended after the ticket was written	Run <code>kb diff</code> , then ask the analyst whether the acceptance criteria still hold. Do not silently implement the new version.
broken	The reference no longer resolves	Escalate. The ticket is not implementable as written.

When a value in your code comes from a standard — a format, an enum, a threshold, a field length — take it from the resolved section at the pinned version and leave a citation comment next to it. Six months later that comment is the only thing connecting the constant to its justification.

7. kb code-ingest — what it extracts

```
kb code-ingest          # defaults are usually right
kb code-ingest --db data/app.sqlite  # name a database explicitly
kb code-ingest --json    # machine-readable report
```

Seven extractors, organised by **artefact kind rather than programming language**, because Strata is adopted across many project lines.

Extractor	Section prefix	Reads
services	svc.<name>	compose files, Dockerfiles, k8s manifests, solution and workspace files
deps	dep.<ecosystem>	manifests for Python, JavaScript, Java, .NET, Go, PHP, Rust, Swift, Dart
commands	cmd.<purpose>	package scripts, Makefiles, tox, shell scripts, CI run steps
tree	struct.tree	the files Git tracks, so an ignored virtualenv never appears
schema	db.<table>	SQL migrations, Prisma, Alembic, EF, and explicitly named SQLite files
integrations	int.<name>	environment-variable keys only
api	api.<tag>	in-repo OpenAPI and Swagger contracts

`tree` always finds something, so the zero-detection rule is: `kb code-ingest` exits `1` when **no extractor other than `tree`** found anything. A document holding only a folder listing is a misconfiguration, not knowledge.

7.1 Two security properties

These are design properties, not conveniences.

1. **Keys, never values.** The only environment file the integrations extractor opens is an `.env.example`, `.sample` or `.template` — never a real `.env` — and its pattern has no capture group around the value at all, so there is nothing to leak whatever a matched line contains. Compose `environment:` blocks *are* read and do hold real inline values, but only the key survives.
2. **Explicit database input.** SQLite schema is read only from paths you name on the command line. A scratch or fixture database in the repository can never become published company knowledge by accident.

7.2 Stated parser limits

These are limits, not bugs.

- `schema` recognises `CREATE TABLE` and `ALTER TABLE ... ADD COLUMN` in sorted filename order. It does not implement a SQL dialect.
- A `CREATE TABLE` for a name already created in a *different* migration directory is dropped with a warning naming both files, and an `ADD COLUMN` applies only inside the directory of its `CREATE TABLE`. Two independent schemas sharing a table name are never merged into one that exists nowhere.
- `RENAME`, `DROP`, `ADD CONSTRAINT` and `ADD INDEX` warn as out of scope.
- A table name outside the recognised identifier shape is skipped **with a warning naming the file**. The reader counts `CREATE TABLE` keywords and compares against what it recognised, so a skip is never silent.
- EF Core migrations are recognised by table name only.
- `build.gradle` is matched by regex, not parsed as a DSL.
- Maven `<plugins>`, `setup.cfg` `extras_require`, `pnpm-workspace.yaml` and compose `healthcheck:` are not read.

7.3 A destination it did not write is refused

If `.kb/<doc_id>/` already holds Markdown or a manifest and that manifest is missing, unreadable, not titled `... -code knowledge`, or lists a section id outside the seven generated prefixes, `kb code-ingest` exits 1 and writes nothing. There is no override flag — move the document or choose another `--doc-id`.

7.4 Determinism and the dirty-tree caveat

Re-running on the same commit on the same platform produces byte-identical files, which is what makes the CI publish a true no-op on an unchanged tree.

Two honest limits: a few glob matches case-normalise per operating system, so the guarantee is per-platform (CI is always Linux, so what gets published is stable); and the manifest's revision derives from the **HEAD commit** while your working tree may hold uncommitted changes. `kb code-ingest` detects that and prints a `dirty_tree` warning.

Local `-code` is what CI would produce if your tracked files were committed now; `kb doctor` warns when your installed strata-kb differs from the version `kb-code.yml` pins.

8. Keeping knowledge current

Neither document needs new per-ticket discipline from you.

8.1 `-code` needs nothing

`-code` is never committed. `.gitignore` carries `.kb/*-code/`, and the document is regenerated from your working tree whenever a command needs it: `kb svc note`, `kb build`, `kb doctor` and `kb publish` refresh it first when it is missing, older than `HEAD`, or your tracked files have changes. A new file counts once it is tracked (`git add`). `kb ticket check` reads the hub copy first — that is what the SA grounded on.

`kb doctor`'s published-snapshot comparison leaves `-code` out entirely (dev repos never match the hub on it); `kb publish` is what actually refreshes it and sends it to the hub.

`kb-code.yml` re-runs `kb code-ingest` → `kb build` → `kb ci-publish` on every push to the default branch, so the hub reflects the current commit once its PR merges.

Trigger	Event	Frequency
Push to default branch, or manual rebuild	<code>code-ingest --build --no-publish</code>	any
Build request	<code>code-ingest --build</code> from <code>strata-kb</code>	never

`-code` has one producer, `kb code-ingest: kb summarize` refuses it. A hub copy that was summarized before 1.4.0 returns to the deterministic L1/L2 on the next CI publish — a one-time diff, not lost information; L3 is unchanged.

`--scaffold-svc` is deliberately never passed in CI. CI must never create `pending` content, since that would fail its own build step.

8.2 `-svc` accrues at handover

`dev-handover` runs, for every service a ticket touched:

```
kb svc note payments-api --ticket ABC-123 \
  --title "Add refund endpoint" --refs "payments-spec $4.2"
```

That appends one row to `hist.<service>`. It is deterministic and **idempotent**: re-running for the same ticket and service updates the row rather than duplicating it.

You never hand-edit `hist.*`. It is an append-only log written only by `kb svc note`. Those sections are facts — "this ticket touched this service" — not claims, so they need no review and never block a build.

`kb svc note` exits 1 on any of: no resolvable repo id; a missing `-svc` or paired `-code` document; an unknown service (a typo must not invent one); an unreadable manifest; or a manifest/file desync.

8.3 Drift in responsibility text

Responsibility text can drift, and the system **self-reports rather than self-heals**.

When `kb code-ingest --scaffold-svc` refreshes a service's code evidence and that section's summary is already `reviewed`, it compares the old and new rendered evidence as a whole, whitespace-trimmed string — not a semantic diff over specific fields. **Any** change beyond leading and trailing whitespace flags `stale-risk: svc.<name>` in the report.

A real change trips it — a file or table added or removed. So does a change in nothing but *how* the evidence is rendered, with no code changed underneath. It is not an error, and it never touches the summary. It is a flag that a human should look.

9. Troubleshooting

Symptom	Cause	Fix
kb code-ingest fails with 1 nothing collected	Only code found anything	Check that your compose file, manifest or CI workflow are where the extractor looks, name a --id if otherwise it's gone
kb code-ingest fails with 1 destination refused	A hard-coded document site or that doc id	Move it, or use --doc-id
kb code-ingest fails with 1 no code section	A publishing section was committed	Finish the seed summary, review, kb commit
kb code-ingest "unknown version"	A bug, or the new version's files are not tracked yet	Copy the id from kb code-ingest --id, add the new file and retry
kb code-ingest "hub code-ingest logs unrecognized"	The last commit publish has not merged on the hub, or the last kb code-ingest --id not found	Open the hub PR named in the Admin job summary, merge it or fix the tag
kb code-ingest "no code yet please -err"	Input configuration lives in the workflow	Move the paths to kb code-ingest --id, kb code-ingest --id, then rerun kb code-ingest --id
kb code-ingest "code-ingest failed to track by git"	A repo from before 1.4.0	git --no-track --add kb code-ingest --id and commit, in one go
kb code-ingest "hub log not pushed - unrecognized" is not set	kb code-ingest was never set in kb code-ingest --id, kb code-ingest --id, before kb code-ingest --id	git commit --no-track --add kb code-ingest --id
kb code-ingest hangs, then fails citing the failure exit	Plain kb code-ingest on a repo whose CI workflow trigger	Use kb code-ingest --id
kb code-ingest refused	This repo is not in the hub's repository-manifest --id	Ask the hub maintainer to add it
kb code-ingest warning	Uncommitted changes while the manifest resolve comes from NCAD	Reinstall locally CI always runs clean
kb code-ingest fails on a Dependabot PR	The hub never self-aligns since required	Expected. Decide whether to keep it required for the PR
kb code-ingest fails kb code-ingest --id	The PR's kb code-ingest section carries no kb code-ingest --id	Run kb code-ingest --id, merge the review and record the result — add a kb code-ingest --id
kb code-ingest fails kb code-ingest --id	The kb code-ingest section says kb code-ingest --id	Fix the BUCKED folder and kb code-ingest --id, kb code-ingest --id
A cluster manifests kb code-ingest --id	The hub published after the ticket was set	kb code-ingest --id, then ask the analyst. Do not merge the CI passed.
MBP server unresponsive	kb code-ingest --id, kb code-ingest --id, kb code-ingest --id	Do not kb code-ingest --id, kb code-ingest --id — it diagnoses which one, and kb code-ingest --id, kb code-ingest --id
kb code-ingest fails with "taken was reported" after the hub maintainer gave you a fresh one	A kb code-ingest --id, kb code-ingest --id, kb code-ingest --id	kb code-ingest --id, kb code-ingest --id, kb code-ingest --id

The hub-lag check itself runs a full `code-ingest` in a temporary worktree on every `kb doctor` — expect it to add real time to the command. On Windows, the same per-platform glob-casing difference noted in §7.4 can make an otherwise-unchanged repo compare as `lags`; a lag verdict that disappears on Linux (CI) is that caveat, not real drift.

10. Command summary

Command	Purpose	Exit
<code>kb mcp-setup [--hub-url URL] [--no-verify]</code>	Write the hub's HTTP MCP credentials into <code>.env</code> and verify them	<code>0</code> ok, <code>1</code> no value/probe failed
<code>kb code-ingest [--db p] [--tags t] [--scaffold-svc] [--json]</code>	Extract code structure into <code>-code</code> (<code>--db/--tags</code> replace <code>code_ingest:</code> for that run)	<code>0</code> ok, <code>1</code> nothing detected or destination refused
<code>kb svc note <svc> --ticket <id> --title "..."</code> <code> [--no-refresh]</code>	Append a row to <code>hist.<svc></code>	<code>0</code> ok, <code>1</code> unknown service or missing document
<code>kb build [--strict] [--no-refresh]</code>	Validate the store	<code>0</code> ok, <code>1</code> error
<code>kb doctor [--no-refresh]</code>	Check KB health (dev repos: refresh <code>-code</code> first)	<code>0</code> ok, <code>1</code> error, <code>2</code> stale citation
<code>kb approve <doc> [--section <id>]</code>	Flip corrected sections to <code>reviewed</code>	<code>0</code> ok
<code>kb publish --pr</code>	Open a hub PR (the seed's first publish)	<code>0</code> ok
<code>kb resolve <file></code>	Check a ticket's citations	<code>0</code> ok, <code>1</code> broken, <code>2</code> stale
<code>kb diff <doc> --against <rev></code>	What an amendment changed	<code>0</code>
<code>kb query <text> / kb get <doc> <section></code>	Search and fetch	<code>0</code>
<code>kb plan waves <plan-file> [--json]</code>	Dependency waves of a <code>docs/impl/<id>-plan.md</code>	<code>0</code> ok, <code>1</code> shared file without a dependency, cycle, unknown task, or missing <code>Depends on:</code>
<code>kb pr lint <file></code>	PR evidence gate	<code>0</code> PASS, <code>1</code> FAIL

Slash commands: `/dev-implement-ticket`, `/dev-design`, `/dev-plan`, `/dev-execute`, `/dev-handover`, `/dev-code-seed`.

11. Reserved names

<repo_id>-code and <repo_id>-svc are reserved for this repository's own code knowledge. A domain document must never take either suffix.

Section prefixes are a published contract that analyst tooling and future tooling key on. Each has exactly one owner:

Prefix	Document	Owner
struct.tree, svc.<name>, db.<table>, dep.<eco>, int.<name>, api.<tag>, cmd.<purpose>	-code	extractor
svc.<name>	-svc	you (model-drafted)
flow.<name>	-svc	you
hist.<name>	-svc	kb svc note only

svc.<name> collides across the two documents **on purpose**. It is the join key that lets a C4 Container(alias, label, technology, description) be filled from -code (the first three) and -svc (the fourth).